Operating Systems — Extra Practice Bank

EXTRA PRACTICE · Weeks 1–6 (Chapters I–III, Practice 1, labs) · 310 questions · Answers at the end

How to use this bank. This is extra practice on top of the main test. Do one part at a time and check yourself with the key at the end. The calculation drills (Part C) are the best preparation for "trace it" questions: do them on paper, frame by frame.

Part	What	Questions
A	Multiple choice, by chapter	184
B	True / False	50
C	Calculation drills: address translation, fragmentation, buddy system, page replacement, process states	42
D	Process or thread? Scenario drills	12
E	Short answer	22

Part A — Multiple Choice

Chapter I — Introduction to OS and System Programming

1. An OS acts as an intermediary between:

A. Users and the computer hardware
B. Two web servers
C. The keyboard and the mouse
D. The CPU and the power supply

2. What is **system software**?

A. Games
B. Word processors
C. Programs designed to operate, control and extend the capabilities of the hardware
D. Web pages

3. Which OS role "allocates and deallocates memory for programs"?

A. Memory management
B. Process management
C. Device management
D. User interface

4. Which OS role "handles file storage, retrieval and permissions"?

A. Process management
B. Device management
C. User interface
D. File system management

5. Which OS role "controls and coordinates hardware components"?

A. User interface
B. Memory management
C. Process management
D. Device management

6. Which OS role "manages the execution of multiple processes"?

A. User interface
B. Memory management
C. Process management
D. File system management

7. What does the **user interface** role do?

A. Lets users interact with the system through a GUI or CLI
B. Allocates memory
C. Controls hardware directly
D. Manages files on disk

8. Linux kernel and Windows NT are examples of:

A. Shells
B. File systems
C. Applications
D. Kernels

9. What does the **shell** do?

A. Interacts directly with the hardware
B. Organizes data storage
C. Takes commands and executes them
D. Allocates memory

10. What does the **file system** do?

A. Runs the CPU
B. Organizes and manages data storage
C. Takes user commands
D. Translates addresses

11. MS-DOS and Windows 95 are which type of OS?

A. Multi-user
B. Single-user
C. Distributed
D. Real-time

12. Linux, Unix and Windows Server are which type?

A. Single-user
B. Multi-user
C. Batch
D. Embedded

13. Which OS type is designed for devices with **limited resources** (for example Embedded Linux)?

A. Embedded
B. Distributed
C. Batch
D. Multi-user

14. Android and iOS are which type?

A. Mobile
B. Batch
C. Real-time
D. Distributed

15. A car's airbag controller must react within milliseconds. Which OS type fits?

A. Batch
B. Real-time
C. Mobile
D. Single-user

16. Payroll jobs run overnight with nobody at the keyboard. Which OS type fits?

A. Real-time
B. Mobile
C. Distributed
D. Batch

17. A smart thermostat has a tiny processor and little memory. Which OS type fits?

A. Batch
B. Distributed
C. Multi-user
D. Embedded

18. Hundreds of students log in to one university server at the same time. Which OS type?

A. Single-user
B. Embedded
C. Multi-user
D. Batch

19. Which is a **similarity** between all OS types in the slides?

A. All have a kernel for hardware control
B. All are real-time
C. All are GUI-only
D. None enforce permissions

20. Which is a **difference** in user interfaces?

A. All OS are GUI-only
B. No OS has a CLI
C. Mobile OS are CLI-only
D. Some OS (servers) are CLI-only

21. Which is a difference in **multitasking**?

A. Batch OS prioritize task scheduling
B. All OS multitask identically
C. Real-time OS prioritize task scheduling
D. Mobile OS cannot multitask

22. Which is a difference in **security**?

A. Single-user OS have the strictest access control
B. No OS enforces permissions
C. Multi-user OS have stricter access control
D. Batch OS have the strictest access control

23. What is the trade-off of a **microkernel**?

A. Always faster, with no downside
B. More robust because services are isolated, but more communication overhead
C. No overhead at all
D. All services run inside the kernel

24. What is the risk of a **monolithic** kernel?

A. It cannot run drivers
B. It has no file system
C. It is always slower than a microkernel
D. A fault in a service inside the kernel can affect everything

25. What is **system programming**?

A. Writing web applications
B. Developing software that provides core services to the hardware and enables application programs to run
C. Designing hardware chips
D. Testing video games

26. The characteristic "close to hardware" refers to:

A. Web design
B. Spreadsheets
C. Memory management, process control and device drivers
D. Marketing software

27. The characteristic "performance-critical" means:

A. Efficient management of CPU, memory and I/O
B. A nice-looking interface
C. Many colours
D. Long source code

28. Which languages are used in system programming?

A. HTML only
B. CSS only
C. SQL only
D. Mainly C, C++ and Rust; sometimes Python for scripting

29. What does `exec()` do?

A. Creates a new process
B. Opens a file
C. Waits for a child
D. Runs a program

30. What does `open()` do?

A. Creates a process
B. Runs a program
C. Opens a file
D. Closes a window

31. What does `fork()` do?

A. Opens a file
B. Creates a process
C. Deletes a file
D. Waits for input

32. What is the purpose of **application** programming?

A. To solve end-user problems
B. To manage hardware and OS resources
C. To write kernels
D. To write device drivers

33. Which is an example of **application** programming?

A. A device driver
B. A kernel
C. Office software
D. A shell

34. The area "system utilities" includes:

A. Kernels
B. Firmware
C. Shells, debuggers (GDB) and process monitors (htop)
D. Sockets

35. The area "embedded systems" includes:

A. Firmware for microcontrollers (Arduino, Raspberry Pi) and RTOS for robotics and cars
B. GDB
C. VPNs
D. Office software

36. The area "networking and security" includes:

A. htop
B. Bash
C. Arduino firmware
D. Sockets, firewalls, VPNs, authentication and encryption

37. What is true about Python's **portability**?

A. It requires recompilation for each OS
B. It is cross-platform
C. It runs only on Windows
D. It runs only on Linux

38. Which is a Python use case named in the slides?

A. Kernel development
B. Real-time systems
C. Scripting, automation, prototyping, and `psutil` process monitoring
D. Device drivers

39. Which is a C use case named in the slides?

A. Rapid prototyping
B. Kernel and driver development, and real-time systems
C. Web pages
D. Spreadsheets

40. In this course, Python is used to:

A. Write a new kernel
B. Replace C completely
C. Build device drivers
D. Observe and script the OS, not to build one

Chapter II — Process and Thread Management

41. What is the difference between a program and a process?

A. They are the same thing
B. A program is a file on disk; a process is that program running, with state
C. A process is stored on disk; a program runs
D. A program is a thread

42. The **text section** of a process contains:

A. The stack
B. The heap
C. The registers
D. The program code

43. The process characteristic "isolation" means:

A. Each process has its own memory space
B. Processes share all memory
C. Processes have no memory
D. Processes share one stack

44. Processes are called **heavyweight** because:

A. They are small
B. They need no resources
C. They share everything
D. They require more system resources

45. What is a **PID**?

A. The next instruction to execute
B. The CPU priority
C. A unique numerical identifier for the process
D. The list of I/O devices

46. What does the **program counter** hold?

A. The process ID
B. The next instruction to execute
C. The priority
D. The I/O status

47. What does the **priority** attribute determine?

A. CPU access priority
B. The list of I/O devices
C. The next instruction
D. The state

48. What does **I/O status** contain?

A. The priority
B. The PID
C. The state
D. The list of I/O devices allocated to the process

49. Which list gives the five process states?

A. Ready, New, Running, Terminated, Waiting
B. New, Running, Ready, Terminated, Waiting
C. Running, New, Waiting, Ready, Terminated
D. New, Ready, Running, Waiting, Terminated

50. A process moves from **new** → **ready** when:

A. It finishes
B. It is admitted
C. It asks for I/O
D. It crashes

51. A process moves from **ready** → **running** when:

A. The scheduler picks it
B. Its I/O finishes
C. It exits
D. It is created

52. A process moves from **running** → **ready** when:

A. It asks for I/O
B. It exits
C. It is created
D. Its time is up or a higher-priority process arrives

53. A process moves from **running** → **terminated** when:

A. It asks for I/O
B. It exits
C. The scheduler picks it
D. Its I/O finishes

54. A **foreground** process:

A. Starts at boot
B. Runs without user interaction
C. Interacts directly with the user
D. Has terminated

55. A **background** process:

A. Runs without user interaction
B. Interacts directly with the user
C. Always starts at boot
D. Is dead

56. A **parent** process:

A. Is created by another process
B. Is dead
C. Has lost its parent
D. Creates other processes

57. A **child** process:

A. Creates other processes
B. Starts at boot
C. Is created by another process
D. Has terminated

58. Which statement about zombies and orphans is correct?

A. They are the same thing
B. A zombie is still running
C. A zombie is dead but not cleaned up; an orphan is alive but has lost its parent
D. An orphan is dead

59. What happens to an orphan process?

A. The OS gives it a new parent
B. It is killed immediately
C. It becomes a zombie forever
D. It becomes a thread

60. Which properties describe threads?

A. Isolated memory
B. Heavyweight
C. Need IPC
D. Shared memory, lightweight, faster context switching, no IPC needed

61. How do threads in one process communicate?

A. Only through IPC
B. Directly through shared memory
C. They cannot communicate
D. Only through the disk

62. Creation overhead: process versus thread?

A. Both low
B. Thread high, process low
C. Process high, thread low
D. Both high

63. Termination: process versus thread?

A. Process slower, thread faster
B. Both the same
C. Thread slower, process faster
D. Neither ever terminates

64. Which single fact explains every row of the process/thread table?

A. Threads are isolated
B. Processes are isolated, threads share
C. Both share all memory
D. Processes are faster

65. What is the slide example of **user-level** threads?

A. A web server handling many requests
B. Solaris
C. The OS scheduler
D. Game-engine threads managing rendering

66. If a **kernel-level** thread blocks:

A. All threads stall
B. The process dies
C. The other threads continue
D. The OS crashes

67. If a **user-level** thread blocks:

A. The whole program can stall
B. The other threads always continue
C. The OS reboots
D. Nothing happens

68. Hybrid threads are managed by:

A. User libraries only
B. The OS only
C. The hardware
D. Both user libraries and the OS

69. In **multiprocessing**, memory is:

A. Shared by default
B. Separate by default
C. Not used
D. Only the heap is shared

70. In **threading**, memory is:

A. Separate
B. Not used
C. Shared: all threads see the same globals
D. Only the stack is shared

71. Threading is best for:

A. I/O-bound tasks, responsive UIs and frequent communication
B. CPU-bound calculation
C. Nothing
D. Creating zombies

72. Threading gives:

A. Always true parallelism
B. No concurrency
C. A need for many computers
D. Concurrency, but not always parallelism

73. A program downloads 100 web pages and mostly waits for the network. Best choice?

A. Multiprocessing on one core only
B. Zombie processes
C. Orphan processes
D. Threading

74. An image filter processes millions of pixels on an 8-core machine. Best choice?

A. Threading
B. A single thread
C. Multiprocessing
D. Zombie processes

75. Multiprocessing is:

A. Heavyweight: slower to create and destroy
B. Lightweight
C. Weightless
D. Exactly the same as threading

76. Process context switching is slower because of:

A. Shared memory
B. Memory isolation
C. Fewer registers
D. Threads

77. Which statement is true?

A. Threads can never crash a process
B. A thread crash only affects its own stack
C. A crash in one thread can bring down the whole process
D. Processes share memory by default

78. A browser that ran every tab as a **thread** would:

A. Be more secure
B. Use more memory
C. Lose fault isolation: one tab crash could kill all tabs
D. Need IPC between tabs

79. Which is described as **lightweight**?

A. Thread
B. Process
C. Zombie
D. Daemon

80. Which situation needs IPC?

A. Two threads in the same process
B. One thread alone
C. A thread and its own stack
D. Two separate processes exchanging data

Practice 1 — Scenarios

81. Scenario 1: why is the OS still responsive while the word processor is frozen?

A. The OS shares the app's memory
B. The OS runs as separate processes in different address spaces, scheduled independently
C. The OS is a thread of the app
D. The app is a zombie

82. Is the frozen word-processor process still alive?

A. No, it has terminated
B. It is a zombie
C. It is an orphan
D. Yes, the OS still tracks it

83. What causes the frozen UI?

A. Not enough disk space
B. A zombie process
C. The UI thread is blocked on synchronous I/O or a long computation
D. A race condition

84. Scenario 2: why does a dead process keep a process-table entry?

A. The entry holds the exit status the parent must read
B. It is still running
C. It still uses CPU
D. It stores the process's files

85. Besides calling `wait()`, what else fixes zombies?

A. Calling `fork()` again
B. Restarting the computer every hour
C. Installing a `SIGCHLD` handler that reaps children
D. Killing the OS

86. Could threads produce zombies in the same way?

A. Yes, exactly the same
B. Only kernel threads
C. Only on Windows
D. No: a finished thread's resources are reclaimed by the process, and exit status is handled at process level

87. What do zombie processes use?

A. A lot of CPU
B. No CPU or memory, but they occupy process-table entries
C. A lot of RAM
D. A lot of disk

88. Scenario 3: closing one Chrome tab kills:

A. Exactly one process
B. All Chrome processes
C. No process
D. The OS

89. Why do the other tabs survive when one tab crashes?

A. Tabs share page tables
B. The OS tears down only that process's memory; the others have separate page tables
C. Chrome restarts the OS
D. The crash is hidden

90. Why can a bad page not read another tab's memory?

A. HTTPS
B. Tabs are threads
C. Each tab has a separate address space
D. Antivirus

91. What overhead does each extra process add?

A. Nothing
B. Its own address space, page tables, file descriptors and kernel structures
C. Only a stack
D. Only registers

92. Scenario 4: a thread switch swaps only:

A. Registers, stack pointer and program counter
B. Page tables
C. The whole address space
D. The file-descriptor table

93. A process switch often:

A. Swaps only registers
B. Does not change anything
C. Changes page tables and flushes the TLB
D. Shares the address space

94. What is the advantage of design B (thread per request)?

A. More resilient
B. Better isolation
C. It never crashes
D. It handles more requests per second

95. What is the advantage of design A (process per request)?

A. It is faster
B. It is more resilient: a crash kills only that process
C. It uses less memory
D. It has no context switches

96. Scenario 5: what are the steps of `counter++`?

A. Load → increment → store
B. Store → load → increment
C. Increment only
D. Load only

97. What is the signature of a race condition?

A. The program runs faster
B. The result changes every run
C. The process becomes a zombie
D. A page fault

98. Which tools does the OS offer against races?

A. Zombies
B. Page faults
C. Mutexes, semaphores and atomic operations
D. The TLB

99. Two **processes**, each with its own private copy of `counter`, each add 1,000,000. Result?

A. Exactly 2,000,000 in one shared counter
B. No race, but each process counts only its own 1,000,000
C. A crash
D. A deadlock

100. Two processes that **share memory** both do `counter++`. Result?

A. The race problem persists
B. It is solved automatically
C. The OS locks the variable for you
D. It is impossible

101. Scenario 6: where do global variables live?

A. On each thread's stack
B. In the registers
C. In thread-local storage
D. In the data segment, shared by all threads

102. Where do local variables live?

A. In the heap
B. In the data segment
C. On each thread's own stack (private)
D. In the code segment

103. Thread-local storage (TLS) is:

A. Shared
B. Private to each thread
C. Stored on disk
D. Kernel-only

104. Between threads, the current working directory is:

A. Shared
B. Private
C. Copied
D. Non-existent

105. Between threads, signal handlers are:

A. Private
B. Shared
C. Copied per thread
D. Non-existent

106. If threads A and B were two **processes**, would B see A's change to a global?

A. Yes, always
B. Yes, if the variable has the same name
C. No; globals are not shared by default, explicit IPC is needed
D. Only on Linux

107. What is the benefit of shared memory between threads?

A. Automatic safety
B. Isolation
C. No bugs
D. Fast communication: no kernel, no copying

108. What is the danger of shared memory between threads?

A. Any thread can corrupt shared data, and synchronization is the programmer's job
B. It is too slow
C. It needs IPC
D. It is impossible

109. Between threads, open file descriptors are:

A. Private
B. Copied
C. Shared
D. Closed

110. In the week 3 dashboard, which choice applies the scenario 1 lesson?

A. Add more `sleep` calls to the UI thread
B. An infinite `while` loop in the UI thread
C. Kill the UI thread
D. Use `root.after()` (or a worker thread) instead of blocking the UI thread

Chapter III — Memory Management

111. What is **memory management**?

A. Deleting files
B. Scheduling the CPU
C. Allocating, tracking and freeing memory so programs run efficiently
D. Printing documents

112. The **protection** goal means:

A. Preventing processes from accessing unauthorized memory
B. Sharing everything
C. Encrypting the disk
D. Compressing memory

113. The **sharing** goal means:

A. No sharing at all
B. Uncontrolled sharing
C. Only disk sharing
D. Controlled memory sharing between processes

114. The **optimization** goal means:

A. Maximizing fragmentation
B. Minimizing fragmentation and maximizing utilization
C. Using only registers
D. Never freeing memory

115. The **allocation** goal means:

A. Preventing unauthorized access
B. Sharing memory
C. Providing memory to processes when needed
D. Compressing files

116. Cache memory is organized into levels named:

A. R1, R2
B. C1, C2
C. M1, M2
D. L1, L2, L3

117. Main memory (RAM) holds:

A. Cache lines only
B. Registers
C. Active programs and data
D. Backup tapes

118. Examples of **secondary** storage:

A. Registers
B. HDDs and SSDs
C. L1 cache
D. Backup tapes only

119. Examples of **tertiary** storage:

A. RAM
B. Registers
C. Backup tapes and optical disks
D. Cache

120. Where does **virtual memory** sit in the hierarchy?

A. Inside the CPU
B. Fastest level
C. Inside the cache
D. Swap space on disk: slowest and lowest cost

121. **Temporal** locality means:

A. The same data is used again soon
B. Nearby data is used soon
C. Data is random
D. Data is on tape

122. Why is faster memory placed closer to the CPU?

A. Cost
B. Response time
C. Capacity
D. Colour

123. Locality of reference justifies:

A. Deleting the cache
B. Removing RAM
C. Keeping frequently used data in fast memory
D. Using tapes for everything

124. In **contiguous** allocation:

A. Each process gets one continuous block
B. A process is split into pages
C. A process is split into segments
D. Memory is never allocated

125. In **fixed** partitioning, memory is divided:

A. On demand
B. Per page
C. Per segment
D. Into fixed-size partitions at boot time

126. In **dynamic** partitioning, memory is:

A. Divided at boot
B. Allocated as needed in variable-sized blocks
C. Never allocated
D. Split into pages

127. What does **compaction** do?

A. Deletes processes
B. Adds RAM
C. Merges free holes into one large block
D. Splits buddies

128. What is **internal** fragmentation?

A. Gaps between allocated blocks
B. Wasted space inside an allocated block
C. Fast memory
D. A page fault

129. What is **external** fragmentation?

A. Waste inside a block
B. Waste in the registers
C. Waste in the cache
D. Free gaps between allocated blocks that are too small to use

130. An 8 MB fixed partition holds a 5 MB process. Result?

A. 3 MB external fragmentation
B. 3 MB internal fragmentation
C. 5 MB internal fragmentation
D. No fragmentation

131. Page size 4 KB, process needs 10 KB. Result?

A. 2 pages, no waste
B. 3 pages, 2 KB external fragmentation
C. 3 pages, 2 KB internal fragmentation
D. 4 pages, 6 KB waste

132. Free holes of 3 MB, 2 MB and 4 MB; a 6 MB process arrives. Result?

A. It does not fit although 9 MB is free: external fragmentation
B. It fits in the 4 MB hole
C. Internal fragmentation of 3 MB
D. It fits because 9 > 6

133. A page table maps:

A. Frame → disk block
B. Segment → base address
C. File → block
D. Page → frame

134. A segment table holds:

A. Page → frame
B. Base address and length
C. The PID
D. The TLB

135. What is an advantage of **segmentation**?

A. It matches the programmer's view and makes sharing (shared libraries) easy
B. No external fragmentation
C. All segments have the same size
D. No table is needed

136. What is a disadvantage of segmentation?

A. Internal fragmentation only
B. No disadvantages
C. External fragmentation and complex management
D. It cannot share code

137. Where is **segmented paging** used?

A. In no modern system
B. Only in batch systems
C. Only on tapes
D. In modern systems such as x86

138. What is a disadvantage of **paging**?

A. Large external fragmentation
B. Small internal fragmentation in the last page, and page-table overhead
C. It cannot support virtual memory
D. It needs contiguous frames

139. In **demand paging**, pages are:

A. All loaded at start
B. Never loaded
C. Loaded randomly
D. Loaded only when needed

140. Page replacement is needed when:

A. The disk is full
B. The CPU is idle
C. No free frame is available
D. The computer first boots

141. FIFO replaces:

A. The least recently used page
B. A random page
C. The page not needed for the longest time
D. The oldest loaded page

142. What is FIFO's performance like?

A. Simple, but poor performance
B. Optimal
C. Needs the future
D. Very expensive

143. What is true about **LRU**?

A. It is the theoretical optimum
B. Near-optimal, but expensive to implement
C. It replaces the oldest page
D. It is random

144. Why does LRU usually beat FIFO?

A. It is random
B. It knows the future
C. Locality of reference
D. It uses more frames

145. In the **buddy system**, block sizes are:

A. Powers of 2
B. Any size
C. Always 1 KB
D. Prime numbers

146. In the buddy system, when a block is freed:

A. It is deleted
B. The disk is compacted
C. It is split again
D. Adjacent free buddies are merged

147. Where is the buddy system used?

A. MS-DOS
B. The Linux kernel
C. Office software
D. The BIOS

148. What is the benefit of **slab allocation**?

A. Large blocks only
B. Power-of-2 splitting
C. No fragmentation for small objects; fast allocation and deallocation
D. Variable-size segments

149. In the comparison table, what fragmentation does slab allocation have?

A. Internal
B. External
C. None
D. Minimal

150. In the comparison table, what fragmentation does the buddy system have?

A. Large external
B. Always 50 % internal
C. None
D. Minimal

151. In the comparison table, what is segmentation's overhead?

A. Low
B. High
C. None
D. Medium

152. In the comparison table, fixed partitioning was used for:

A. Modern OS
B. Kernel objects
C. Early OS
D. Program memory

153. Page size 1024, logical address 2100, page 2 → frame 7. Physical address?

A. 7220
B. 2100
C. 7168
D. 9268

154. Page size 4096, logical address 8200. Page number and offset?

A. Page 1, offset 4104
B. Page 2, offset 100
C. Page 3, offset 8
D. Page 2, offset 8

155. Page size 512, logical address 1500. Page number and offset?

A. Page 3, offset 0
B. Page 2, offset 476
C. Page 2, offset 488
D. Page 1, offset 988

156. During address translation, the offset:

A. Doubles
B. Becomes the frame number
C. Never changes
D. Is discarded

157. Reference string 1, 2, 3, 1, 4, 1, 2 with 3 frames. How many faults with **LRU**?

A. 5
B. 6
C. 4
D. 7

158. Same string with **Optimal**?

A. 6
B. 5
C. 3
D. 4

159. Same string with **FIFO**?

A. 5
B. 6
C. 4
D. 7

160. Buddy system, 1024 KB free, request 100 KB. Which block is allocated?

A. 100 KB
B. 256 KB
C. 128 KB
D. 64 KB

161. Which Python module detects memory leaks in the week 5 lab?

A. `tracemalloc`
B. `psutil`
C. `platform`
D. `tkinter`

162. `snap2.compare_to(snap1, 'lineno')` shows:

A. CPU usage
B. The list of files
C. The number of threads
D. The source lines that allocated the most new memory

Labs — Reading the Code

163. What does `platform.system()` return?

A. CPU usage
B. The OS name, such as "Windows" or "Linux"
C. The PID
D. A list of files

164. What does `platform.release()` return?

A. The OS release / version
B. The machine name
C. The PID
D. The memory usage

165. What does `platform.node()` return?

A. The number of CPUs
B. The RAM size
C. The OS name
D. The machine's network name

166. What does `os.listdir(".")` do?

A. Deletes files
B. Creates a directory
C. Asks the file system for the entries in the current directory
D. Shows CPU usage

167. `open(name, "w")` followed by `f.write(...)`:

A. Creates a file and writes to it, using the `open` system call underneath
B. Reads a file only
C. Deletes a file
D. Starts a new process

168. What does `psutil.cpu_percent()` return?

A. Memory usage
B. Current CPU usage
C. Disk usage
D. The PID

169. What does `psutil.virtual_memory().percent` return?

A. CPU usage
B. The name of the swap file
C. The PID
D. The percentage of memory in use

170. What does `deque(maxlen=60)` do in the dashboard?

A. Sleeps for 60 seconds
B. Starts 60 threads
C. Keeps only the last 60 readings
D. Starts 60 processes

171. A `while` loop with `time.sleep(1)` in the tkinter UI thread would:

A. Freeze the window
B. Make the window faster
C. Create a new process
D. Free memory

172. In the paging lab, `program_pages = ['A', 'B', 'C', 'D']` are:

A. Physical frames
B. The program's logical pages 0–3
C. Processes
D. Segments

173. `page_table = {}` will map:

A. Frame → page
B. Process → PID
C. File → block
D. Logical page → physical frame

174. What does `random.choice(available_frames)` show?

A. Allocation must be contiguous
B. Pages must be adjacent
C. Any free frame will do: allocation is non-contiguous
D. Only frame 0 can be used

175. Only one frame is free and the program loads 4 pages. What happens?

A. Page A is loaded; B, C, D are silently skipped; a later access to a missing page raises `KeyError`
B. All four pages are loaded
C. The program crashes immediately
D. Pages share the one frame

176. `page_table = {0: 4, 1: 0, 2: 7, 3: 2}` and memory `['B','OS','D','AppX','A',None,'OS',None]`. What does page 3 hold?

A. `'A'`
B. `'B'`
C. `'OS'`
D. `'D'`

177. Same example: what does page 1 hold?

A. `'B'`
B. `'A'`
C. `'D'`
D. `None`

178. `logical_address = 5` with pages 0–3. Result?

A. Returns `None`
B. `KeyError: 5`, an invalid address
C. Frame 5
D. `'C'`

179. Free frames are 0, 2, 4, 5, 7. With first-fit (`available_frames[0]`), which frames do pages 0–3 get?

A. 7, 5, 4, 2
B. 1, 3, 6, 7
C. 0, 2, 4, 5
D. 0, 1, 2, 3

180. Does random choice or first-fit cause external fragmentation in paging?

A. Yes, random does
B. Yes, first-fit does
C. Both do
D. No; all frames are the same size and pages need not be adjacent

181. What happens with `available_frames[-1]` instead of `random.choice`?

A. Identical output every run
B. A different result every run
C. A `KeyError`
D. Pages overlap

182. A program has pages 0–3. Is "logical page 4" valid because frame 4 exists?

A. Yes, frame 4 exists
B. No; logical page numbers and physical frame numbers are different number spaces
C. It maps to frame 4 automatically
D. It maps to the OS frame

183. The slide code declares `spage_table = {}` but then uses `page_table`. If you ran it unchanged, what would happen?

A. `NameError`: `page_table` is not defined
B. `KeyError`
C. It works fine
D. The pages go to the OS frames

184. In the unmodified lab code, can two pages land in the same frame?

A. Yes, always
B. Yes, sometimes
C. No; the frame is removed from `available_frames` after use
D. Only page 0

Part B — True / False

Write **T** or **F**. If false, correct the statement.

1. The kernel is the user interface that takes commands.
2. A process is an instance of a program in execution.
3. MS-DOS is a multi-user OS.
4. QNX is an example of a real-time OS.
5. Linux uses a microkernel.
6. System calls let programs request services from the kernel.
7. Python is compiled and faster than C.
8. C programs need recompiling for each OS.
9. A zombie process is still running.
10. An orphan process is still alive.
11. A daemon is a background service that starts at boot.
12. When its I/O finishes, a waiting process goes straight to running.
13. Threads in the same process share the heap.
14. Each thread has its own stack.
15. Threads need IPC to communicate.
16. Process context switches are faster than thread switches.
17. If one user-level thread blocks, the others always keep running.
18. Multiprocessing gives true parallelism on multiple cores.
19. Threading is best for CPU-bound tasks.
20. A frozen UI always means the process has crashed.
21. Zombie processes use a lot of CPU.
22. Calling `wait()` or `waitpid()` reaps zombie children.
23. Chrome uses separate processes for tabs to get fault isolation.
24. In a thread-per-request server, one bad pointer can crash every request.
25. `counter++` is an atomic operation.
26. A race condition gives the same wrong result every run.
27. The OS automatically protects shared variables.
28. Global variables live in the data segment.
29. Local variables are shared between threads.
30. Two processes see each other's global variables by default.
31. CPU registers are the fastest memory.
32. Going down the memory hierarchy, memory gets faster.

33. Temporal locality means using nearby data.

34. Fixed partitioning causes internal fragmentation.

35. Dynamic partitioning causes external fragmentation.

36. Paging causes external fragmentation.

37. Segmentation can cause external fragmentation.

38. A page and a frame have the same size.

39. The offset changes during address translation.

40. A page fault is an error that crashes the program.

41. FIFO replaces the least recently used page.

42. Optimal page replacement needs knowledge of the future.

43. LRU always gives fewer faults than FIFO.

44. The buddy system uses power-of-2 block sizes.

45. Slab allocation is used for kernel objects.

46. Virtual memory uses part of secondary storage to look like main memory.

47. Compaction fixes internal fragmentation.

48. In the paging lab, random frame choice shows that paging works with any free frame.

49. Logical page numbers and physical frame numbers are the same number space.

50. `random.seed(42)` makes the paging lab reproducible.

Part C — Calculation Drills

C1. Address translation

Use this page table for questions 1–10: **page 0 → frame 3, page 1 → frame 6, page 2 → frame 1, page 3 → frame 4, page 4 → frame 0.** For each logical address give the **page number, offset and physical address**.

#	Page size	Logical address
1	1024	500
2	1024	1024
3	1024	2500
4	1024	3500
5	1024	4100
6	1024	5200
7	2048	3000
8	2048	6000
9	4096	9000
10	512	1300

11. Reverse: page size 1024, same page table. Physical address 6200. Which logical address produced it?

C2. Fragmentation and the buddy system

1. A 10 MB fixed partition holds a 7 MB process. Fragmentation type and amount?
2. Page size 4 KB, process 13 KB. Pages needed and wasted space?
3. Page size 2 KB, process 9 KB. Pages needed and wasted space?
4. Page size 4 KB, process 16 KB. Pages needed and wasted space?
5. Free holes: 5 MB, 3 MB, 6 MB. An 8 MB process arrives. Does it fit? What is the problem and the fix?
6. Same holes. A 6 MB process arrives. Does it fit?
7. Buddy system, 1024 KB free, request 200 KB. Block size and waste?
8. Buddy system, 1024 KB free, request 33 KB. Block size and waste?
9. Buddy system, 1024 KB free, request 512 KB. Block size and waste?
10. Buddy system, 1024 KB free, request 130 KB. Show the splits, the block allocated and the free blocks left.

C3. Page replacement traces

For each trace, fill the frames step by step and count the page faults for **FIFO**, **LRU** and **Optimal**.

1. **Trace 1:** `7, 0, 1, 2, 0, 3, 0, 4` with **3 frames**
2. **Trace 2:** `1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5` with **3 frames**
3. **Trace 3:** `2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2` with **3 frames**
4. **Trace 4:** `5, 1, 5, 2, 1, 3` with **2 frames**
5. **Challenge (not on the slides):** Trace 2 again with **FIFO and 4 frames**. Compare with the 3-frame FIFO result. What is surprising?

C4. Process-state transitions

Give the transition (from → to) for each event.

1. The scheduler picks a ready process.
2. The process asks to read a file from disk.
3. The disk read completes.
4. The process's time slice runs out.
5. A program is started from the shell and admitted.
6. The process calls `exit`.
7. A higher-priority process arrives while this one is running.
8. The process waits for keyboard input.

Part D — Process or Thread? Scenario Drills

For each scenario, say whether it is a **process-level** or **thread-level** matter (or name the mechanism), and give one sentence of reasoning.

1. A music player's window freezes while it scans a 50 GB library, but other apps still work.
2. A bank wants a crash in one customer's session never to affect another customer's session.
3. A program must add up a billion numbers as fast as possible on 8 cores.

4. A chat app must download images while keeping its interface responsive.
5. Ten threads update a shared balance; the final balance is wrong and different every run.
6. `ps` shows many processes in state `Z`, and new programs fail to start.
7. A parent process is killed, but its child keeps running.
8. A web server must handle 10,000 small connections per second with low overhead and still survive crashes.
9. Two separate programs need to exchange data.
10. Thread B cannot see a variable declared inside a function that thread A is running.
11. Each browser tab appears as a separate entry in Task Manager.
12. A log-cleaning service starts automatically every time the server boots.

Part E — Short Answer

1. Define an operating system and list its five main roles.
2. Explain the kernel, the shell and the file system, with one example each.
3. Name the seven OS types from the slides and give one example of each.
4. Compare a monolithic kernel and a microkernel.
5. What is a system call? Name the three examples from the slides and what they do.
6. Compare system programming and application programming in four aspects.
7. Compare Python and C for system work.
8. Explain the difference between a program and a process, and list five process attributes.
9. Describe the five process states and the transitions between them. Why does a process go from waiting to **ready**, not to running?
10. Compare zombie and orphan processes. How do you fix zombies?
11. Give six differences between processes and threads.
12. Compare user-level, kernel-level and hybrid threads.
13. Compare multiprocessing and threading. When should you use each?
14. Explain why two threads doing `counter++` lose increments, and how to fix it.
15. List what threads share and what each thread keeps private.
16. Explain internal and external fragmentation with one numeric example each.
17. Compare paging and segmentation.
18. Explain virtual memory, demand paging and a page fault.
19. Compare FIFO, LRU and Optimal page replacement.
20. Compare the buddy system and slab allocation.
21. In the paging lab, what happens if only one frame is free when the program loads four pages? What would a real OS do?
22. Why does it not matter for paging whether frames are chosen randomly or first-fit?

Answer Key

Part A

Chapter I: 1 A · **2** C · **3** A · **4** D · **5** D · **6** C · **7** A · **8** D · **9** C · **10** B · **11** B · **12** B · **13** A · **14** A · **15** B · **16** D · **17** D · **18** C · **19** A · **20** D · **21** C · **22** C · **23** B · **24** D · **25** B · **26** C · **27** A · **28** D · **29** D · **30** C · **31** B · **32** A · **33** C · **34** C · **35** A · **36** D · **37** B · **38** C · **39** B · **40** D

Chapter II: 41 B · **42** D · **43** A · **44** D · **45** C · **46** B · **47** A · **48** D · **49** D · **50** B · **51** A · **52** D · **53** B · **54** C · **55** A · **56** D · **57** C · **58** C · **59** A · **60** D · **61** B · **62** C · **63** A · **64** B · **65** D · **66** C · **67** A · **68** D · **69** B · **70** C · **71** A · **72** D · **73** D · **74** C · **75** A · **76** B · **77** C · **78** C · **79** A · **80** D

Practice 1: 81 B · **82** D · **83** C · **84** A · **85** C · **86** D · **87** B · **88** A · **89** B · **90** C · **91** B · **92** A · **93** C · **94** D · **95** B · **96** A · **97** B · **98** C · **99** B · **100** A · **101** D · **102** C · **103** B · **104** A · **105** B · **106** C · **107** D · **108** A · **109** C · **110** D

Chapter III: 111 C · **112** A · **113** D · **114** B · **115** C · **116** D · **117** C · **118** B · **119** C · **120** D · **121** A · **122** B · **123** C · **124** A · **125** D · **126** B · **127** C · **128** B · **129** D · **130** B · **131** C · **132** A · **133** D · **134** B · **135** A · **136** C · **137** D · **138** B · **139** D · **140** C · **141** D · **142** A · **143** B · **144** C · **145** A · **146** D · **147** B · **148** C · **149** C · **150** D · **151** B · **152** C · **153** A · **154** D · **155** B · **156** C · **157** A · **158** D · **159** B · **160** C · **161** A · **162** D

Working for the calculations: **153** 7 × 1024 + 52 = 7220 (2100 − 2048 = 52). **154** 8200 ÷ 4096 = 2, 8200 − 8192 = 8. **155** 1500 ÷ 512 = 2, 1500 − 1024 = 476. **157–159** see the study guide trace: LRU 5, Optimal 4, FIFO 6. **160** 1024 → 512 → 256 → 128; 128 is the smallest power of 2 ≥ 100.

Labs: 163 B · **164** A · **165** D · **166** C · **167** A · **168** B · **169** D · **170** C · **171** A · **172** B · **173** D · **174** C · **175** A · **176** D · **177** A · **178** B · **179** C · **180** D · **181** A · **182** B · **183** A · **184** C

Part B — True / False

#		Correction / reason
1	F	That is the **shell**. The kernel is the core that talks to hardware.
2	T	
3	F	MS-DOS is **single-user**.
4	T	VxWorks and QNX are the RTOS examples.
5	F	Linux is **monolithic**; QNX is the microkernel example.
6	T	e.g. `fork()`, `exec()`, `open()`.
7	F	Python is interpreted and slower; C is compiled and faster.
8	T	Python is the cross-platform one.
9	F	A zombie has **terminated**; only its table entry remains.
10	T	Its parent is gone; the OS gives it a new parent.
11	T	
12	F	Waiting → **ready**; the scheduler then picks it.
13	T	Code, globals, heap, open files, cwd and signal handlers are shared.
14	T	Stack, registers, PC and TLS are private.
15	F	Threads use shared memory; **processes** need IPC.
16	F	Thread switches are faster: no page-table switch.
17	F	User-level threads are invisible to the OS; one block can stall the whole program.
18	T	
19	F	Threading suits **I/O-bound** work; CPU-bound → multiprocessing.
20	F	Usually the **UI thread is blocked**; the process is alive.
21	F	They use no CPU or memory, but fill the finite process table.
22	T	Or a `SIGCHLD` handler.
23	T	Cost: resource overhead.
24	T	All threads share one address space.
25	F	Load → increment → store: three steps.
26	F	The result **changes every run**.
27	F	The programmer must use mutexes, semaphores or atomics.
28	T	
29	F	Locals are on each thread's private **stack**.
30	F	Not shared by default; explicit IPC is needed.
31	T	
32	F	Slower, bigger, cheaper per bit.
33	F	Temporal = the **same** data again; spatial = nearby data.
34	T	
35	T	Fixed with compaction.

#		Correction / reason
36	F	Paging has **no** external fragmentation (small internal in the last page).
37	T	Variable-size segments leave gaps.
38	T	
39	F	The offset never changes; only page → frame.
40	F	It is a normal signal: the OS loads the page.
41	F	FIFO replaces the **oldest loaded** page; LRU replaces the least recently used.
42	T	It is a theoretical benchmark.
43	F	Usually, not always (see Trace 2 and Trace 4 in Part C).
44	T	
45	T	
46	T	Usually through demand paging.
47	F	Compaction fixes **external** fragmentation.
48	T	
49	F	They are different number spaces linked by the page table.
50	T	

Part C

C1. Address translation

#	Page	Offset	Frame	Physical address
1	0	500	3	3 × 1024 + 500 = **3572**
2	1	0	6	6 × 1024 + 0 = **6144**
3	2	452	1	1024 + 452 = **1476**
4	3	428	4	4096 + 428 = **4524**
5	4	4	0	0 + 4 = **4**
6	5	80	—	**Invalid**: page 5 is not in the page table (in the lab this is a `KeyError`)
7	1	952	6	6 × 2048 + 952 = **13240**
8	2	1904	1	2048 + 1904 = **3952**
9	2	808	1	4096 + 808 = **4904**
10	2	276	1	512 + 276 = **788**

11. 6200 ÷ 1024 = frame 6, offset 6200 − 6144 = 56. Frame 6 belongs to page 1, so logical address = 1 × 1024 + 56 = **1080**.

C2. Fragmentation and buddy

1. **3 MB internal** fragmentation (waste inside the partition).
2. 13 ÷ 4 → **4 pages** = 16 KB, **3 KB internal** in the last page.
3. 9 ÷ 2 → **5 pages** = 10 KB, **1 KB internal**.
4. **4 pages** exactly, **0 waste**.

5. **No.** 14 MB is free in total, but no single hole is 8 MB: **external fragmentation**. Fix: **compaction** (merge the holes), or use paging.
6. **Yes**, it fits exactly in the 6 MB hole.
7. **256 KB** block (1024 → 512 → 256), waste **56 KB** (internal).
8. **64 KB** block, waste **31 KB**.
9. **512 KB** block, waste **0**.
10. Split 1024 → two 512s; split one 512 → two 256s. Allocate **one 256 KB** block (130 KB used, 126 KB wasted). Free afterwards: **one 512 KB and one 256 KB** buddy.

C3. Page replacement (frames listed after each reference; F = fault, · = hit)

Trace 1 — `7 0 1 2 0 3 0 4`, 3 frames

Ref	7	0	1	2	0	3	0	4	Faults
FIFO	7	7 0	7 0 1	2 0 1	2 0 1	2 3 1	2 3 0	4 3 0	
	F	F	F	F	·	F	F	F	**7**
LRU	7	7 0	7 0 1	2 0 1	2 0 1	2 0 3	2 0 3	4 0 3	
	F	F	F	F	·	F	·	F	**6**
Optimal	7	7 0	7 0 1	2 0 1	2 0 1	2 0 3	2 0 3	4 0 3	
	F	F	F	F	·	F	·	F	**6**

Six different pages appear, so 6 faults is the minimum; LRU happens to reach it here. FIFO loses page 0 at the 6th reference because it is the oldest, even though it is in heavy use.

Trace 2 — `1 2 3 4 1 2 5 1 2 3 4 5`, 3 frames

Ref	1	2	3	4	1	2	5	1	2	3	4	5	Faults
FIFO	1	1 2	1 2 3	4 2 3	4 1 3	4 1 2	5 1 2	5 1 2	5 1 2	5 3 2	5 3 4	5 3 4	
	F	F	F	F	F	F	F	·	·	F	F	·	**9**
LRU	1	1 2	1 2 3	4 2 3	4 1 3	4 1 2	5 1 2	5 1 2	5 1 2	3 1 2	3 4 2	3 4 5	
	F	F	F	F	F	F	F	·	·	F	F	F	**10**
Optimal	1	1 2	1 2 3	1 2 4	1 2 4	1 2 4	1 2 5	1 2 5	1 2 5	3 2 5	4 2 5	4 2 5	
	F	F	F	F	·	·	F	·	·	F	F	·	**7**

Here **LRU (10) is worse than FIFO (9)**: "LRU usually beats FIFO" is a tendency, not a guarantee. Optimal (7) is still the lower bound.

Trace 3 — `2 3 2 1 5 2 4 5 3 2 5 2`, 3 frames

Ref	2	3	2	1	5	2	4	5	3	2	5	2	Faults
FIFO	2	2 3	2 3	2 3 1	5 3 1	5 2 1	5 2 4	5 2 4	3 2 4	3 2 4	3 5 4	3 5 2	
	F	F	·	F	F	F	F	·	F	·	F	F	**9**
LRU	2	2 3	2 3	2 3 1	2 5 1	2 5 1	2 5 4	2 5 4	3 5 4	3 5 2	3 5 2	3 5 2	
	F	F	·	F	F	·	F	·	F	F	·	·	**7**
Optimal	2	2 3	2 3	2 3 1	2 3 5	2 3 5	4 3 5	4 3 5	4 3 5	2 3 5	2 3 5	2 3 5	
	F	F	·	F	F	·	F	·	·	F	·	·	**6**

Optimal 6 ≤ LRU 7 ≤ FIFO 9: the usual order, thanks to locality.

Trace 4 — `5 1 5 2 1 3`, 2 frames

Ref	5	1	5	2	1	3	Faults
FIFO	5	5 1	5 1	2 1	2 1	2 3	
	F	F	·	F	·	F	**4**
LRU	5	5 1	5 1	5 2	1 2	1 3	
	F	F	·	F	F	F	**5**
Optimal	5	5 1	5 1	2 1	2 1	3 1	
	F	F	·	F	·	F	**4**

Again FIFO beats LRU on this short string. At reference 4 LRU throws out page 1 (used less recently than 5), but page 1 is needed next.

Trace 5 (challenge) — FIFO with 4 frames on Trace 2: frames `1 | 1 2 | 1 2 3 | 1 2 3 4 | · | · | 5 2 3 4 | 5 1 3 4 | 5 1 2 4 | 5 1 2 3 | 4 1 2 3 | 4 5 2 3` → **10 faults**, more than the **9** with 3 frames. **More frames gave more faults.** This is called *Belady's anomaly*; it can happen with FIFO but not with LRU or Optimal. (Not on the slides; extra knowledge.)

C4. Process states

1. Ready → running · 2. Running → waiting · 3. Waiting → ready · 4. Running → ready · 5. New → ready · 6. Running → terminated · 7. Running → ready · 8. Running → waiting

Part D — Process or thread?

1. **Thread-level.** The process is alive; its UI thread is blocked. Move the scan to a background thread.
2. **Processes.** Separate address spaces give fault isolation (the Chrome reasoning).
3. **Multiprocessing.** CPU-bound work needs true parallelism on several cores.
4. **Threads.** I/O-bound work plus a responsive UI: one UI thread, one or more download threads.
5. **Race condition (thread-level).** Unsynchronized `balance += …` on shared memory; fix with a mutex or atomic operation.
6. **Zombies (process-level).** The parent does not call `wait()` / `waitpid()`; the finite process table fills up.
7. **Orphan (process-level).** The child is still alive; the OS assigns it a new parent.
8. **Hybrid**: a process pool where each process runs a thread pool: threads for speed, processes for crash isolation.
9. **IPC (process-level).** Separate processes do not share memory by default: use pipes, shared memory or message queues.
10. **Thread-level.** The variable is a local on A's private stack; only globals/heap are shared.
11. **Processes** (one per tab) for isolation and security; the cost is overhead.
12. **Daemon process**: a background service that starts at boot.

Part E — Model answers

1. An OS is software that manages hardware and software resources and provides common services for programs; it is the intermediary between users and hardware. Roles: process management, memory management, file system management, device management, user interface.
2. **Kernel**: the core, talks directly to hardware (Linux kernel, Windows NT). **Shell**: interface that takes and executes commands (Bash, PowerShell). **File system**: organizes data storage (NTFS, FAT32).

3. Single-user (MS-DOS, Windows 95), multi-user (Linux, Unix, Windows Server), batch (IBM OS), real-time (VxWorks, QNX), distributed (Amoeba, Plan 9), embedded (Embedded Linux), mobile (Android, iOS).
4. **Monolithic** (Linux): most services such as drivers and file systems run inside the kernel; fast, but a fault there affects everything. **Microkernel** (QNX): minimal kernel, services run as user-space processes; more robust, more communication overhead.
5. A system call is how a program asks the kernel to do something it cannot do itself. `fork()` creates a process, `exec()` runs a program, `open()` opens a file.
6. Abstraction: low level vs high level. Languages: C, C++, Assembly, Rust vs Python, Java, JavaScript, C#. Purpose: manage hardware and OS resources vs solve end-user problems. Examples: kernels, drivers, shells vs browsers, games, office software.
7. Python: high-level, easy, slower (interpreted), rich libraries, cross-platform; used for scripting and monitoring (`psutil`). C: low-level with manual memory management, faster (compiled, direct hardware access), needs recompiling per OS; used for kernels, drivers and real-time systems.
8. A program is a file on disk; a process is that program in execution with state. Attributes: PID, program counter, state, priority, I/O status.
9. New → ready (admitted) → running (scheduler picks it) → waiting (needs I/O) / ready (time up or higher priority) / terminated (exits). When I/O finishes the process goes to **ready** because the CPU may be busy; it must wait for the scheduler to pick it again.
10. A **zombie** has terminated but its parent has not collected the exit status, so its table entry remains. An **orphan** is alive but its parent terminated; the OS gives it a new parent. Fix zombies: the parent calls `wait()`/`waitpid()` or installs a `SIGCHLD` handler.
11. Memory: isolated vs shared. Creation overhead: high vs low. Communication: IPC vs direct shared memory. Context switching: slower vs faster. Resource usage: more vs less. Termination: slower vs faster.
12. **User-level**: managed by a user library, fast, invisible to the OS; one block can stall the whole program (game-engine rendering). **Kernel-level**: managed by the OS, slower but if one blocks others continue (web server). **Hybrid**: both combined (Solaris).
13. **Multiprocessing**: separate memory, heavyweight, true parallelism; use for CPU-bound calculation. **Threading**: shared memory, lightweight, concurrency; use for I/O-bound work, responsive UIs and frequent communication.
14. `counter++` is load → increment → store. Two threads can load the same value, both add 1 and both store it, so one increment is lost; the result is too low and changes every run (race condition). Fix: protect the increment with a mutex, or use an atomic operation.
15. **Shared**: code segment, data segment (globals), heap, open file descriptors, current working directory, signal handlers. **Private**: stack, registers, program counter, thread-local storage.
16. **Internal**: waste inside an allocated block, e.g. a 5 MB process in an 8 MB fixed partition wastes 3 MB. **External**: free gaps between blocks too small to use, e.g. holes of 3, 2 and 4 MB cannot hold a 6 MB process although 9 MB is free.
17. **Paging**: fixed-size pages/frames, page table, no external fragmentation, small internal waste in the last page. **Segmentation**: variable-size segments matching program structure (code, data, stack), segment table with base and length, easy sharing, but external fragmentation.
18. Virtual memory uses part of secondary storage to give the illusion of more main memory, so more programs can run. With demand paging, a page is loaded only when it is first needed; touching a page that is not in memory causes a **page fault**, a normal event in which the OS loads the page (replacing another if no frame is free).

19. **FIFO** replaces the oldest loaded page: simple but poor. **LRU** replaces the least recently used: near-optimal, expensive. **Optimal** replaces the page not needed for the longest time: needs the future, so it is only a benchmark.

20. **Buddy**: power-of-2 blocks; split larger blocks into buddies to fit, merge free buddies back; minimal fragmentation; used in the Linux kernel. **Slab**: pre-allocated slabs for frequently used kernel objects; fast, no fragmentation for small objects.

21. `if available_frames:` is true only once: page A is loaded and B, C, D are silently skipped; a later access to a missing page raises `KeyError`. A real OS would be out of free frames and would swap pages to disk (page replacement).

22. Every frame is the same size and pages do not need to be adjacent, so any page fits in any free frame. Choosing randomly or first-fit changes which frames are used, but never causes external fragmentation.